DEPO / INDUSTRIAL DATA PLATFORM

Enterprise software architecture

Ontology alignment, governed agents and reusable data products

C4 • UML • TOGAF-aligned architecture views

16 October 2026 | Repository baseline 0d728b9

Architecture scope and TOGAF-aligned layers

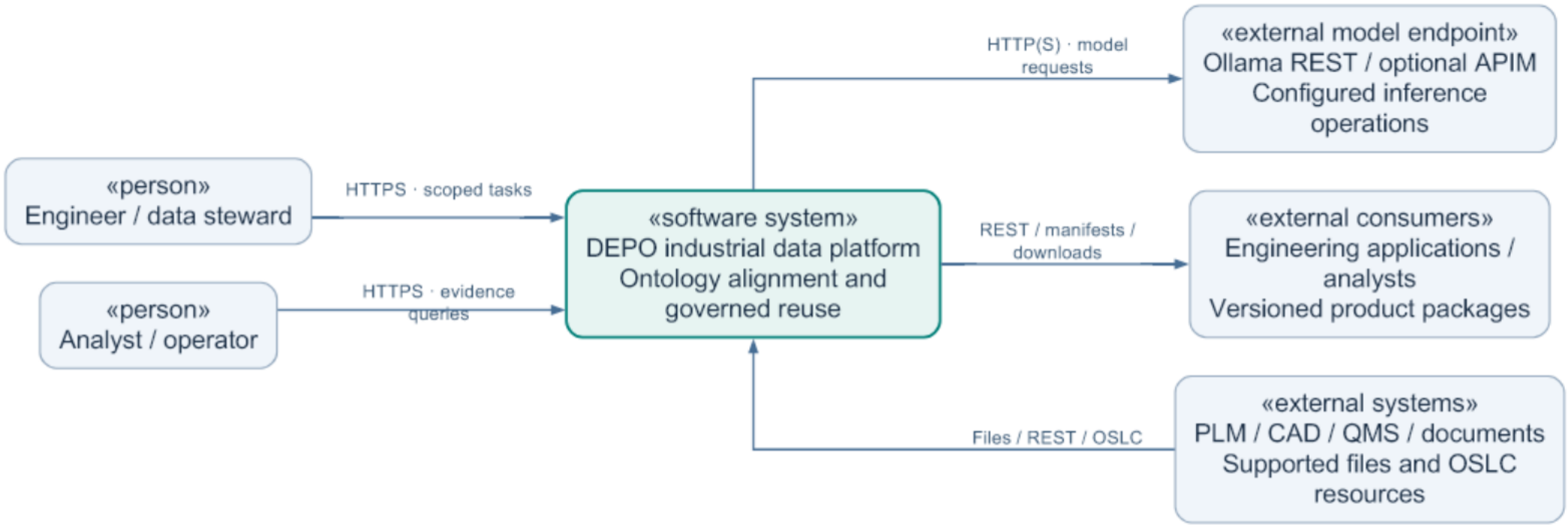
Four architecture domains organize the views; this is an implementation architecture, not a certification claim.

Layer	Architecture responsibility	DEPO implementation
Business	Engineering meaning, stewardship and reusable evidence	Ontology review, instance linking, governed products
Application	Service responsibilities and permitted interactions	10 APIs, 3 workers, React / Siemens IX frontend
Data	Identity, lineage, persistence and publication outcomes	PostgreSQL, Neo4j and immutable artifact references
Technology	Runtime, network, security and operational readiness	Windows deployment, optional gateway, Spark and Ollama

Scope: repository capabilities and customer deployment controls; untested external connectors remain deployment work.

C4 context — users, platform and external systems

Level 1 · DEPO is the system of interest; external systems remain outside its authority.



WHAT Industrial semantic integration and reuse.

WHY Preserve meaning and evidence across systems.

WHO Engineers, stewards, analysts and operators.

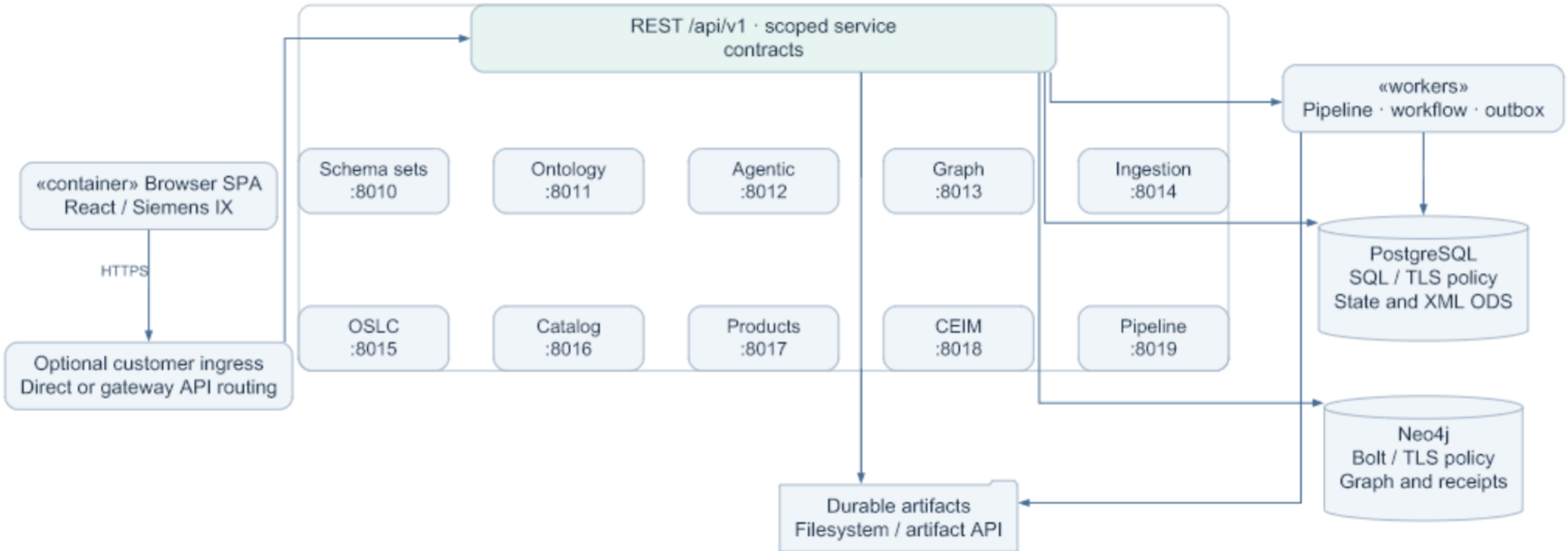
WHEN Import, align, query and publish tasks.

WHERE Customer application and private data services.

HOW Scoped APIs, supported files and model REST.

C4 containers — ten APIs and three workers

Level 2 · Container names and ports follow the deployment manifest.



WHAT Deployable APIs, workers and persistent stores.

WHY Separate request handling from durable execution.

WHO Frontend, domain services and worker processes.

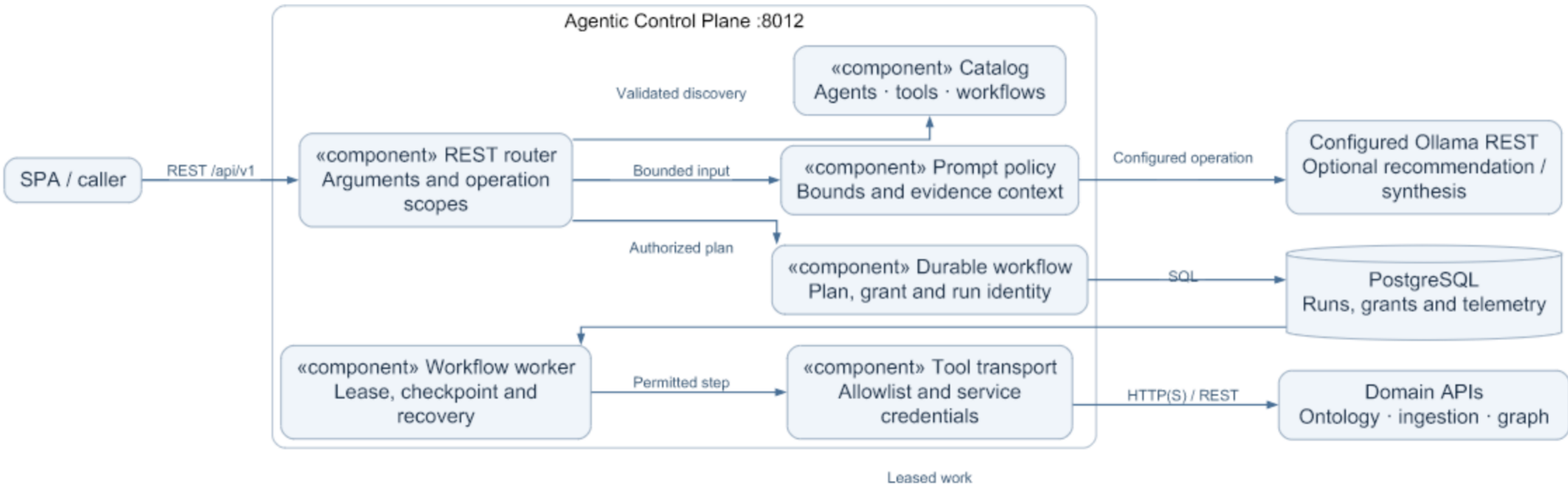
WHEN Interactive requests and queued background work.

WHERE Internal ports 8010–8019; public routing is configurable.

HOW REST, PostgreSQL leases, artifacts and Bolt.

C4 components — agentic execution boundary

Level 3 · Recommendations and durable workflows share the validated tool catalog.



WHAT Agent discovery, planning and permitted tool execution.

WHY Keep model suggestions behind service authorization.

WHO Agentic router, workflow worker and domain APIs.

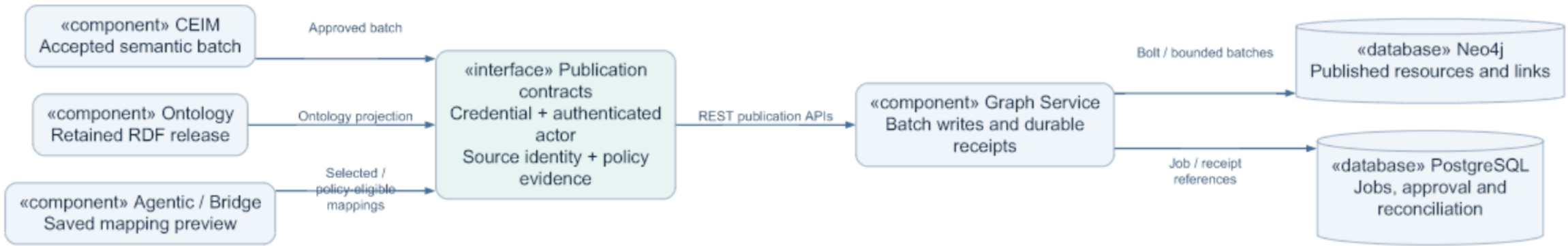
WHEN Recommendation and approved multi-step workflows.

WHERE Agentic :8012 with PostgreSQL run state.

HOW Validated catalog, grants, leases and bounded tools.

UML components — governed graph publication

Ontology projection, CEIM batches and Bridge links have separate contracts.



WHAT Distinct publication contracts converge on graph writes.

WHEN After source and policy evidence satisfy a contract.

WHY Prevent hidden writes and ambiguous completion.

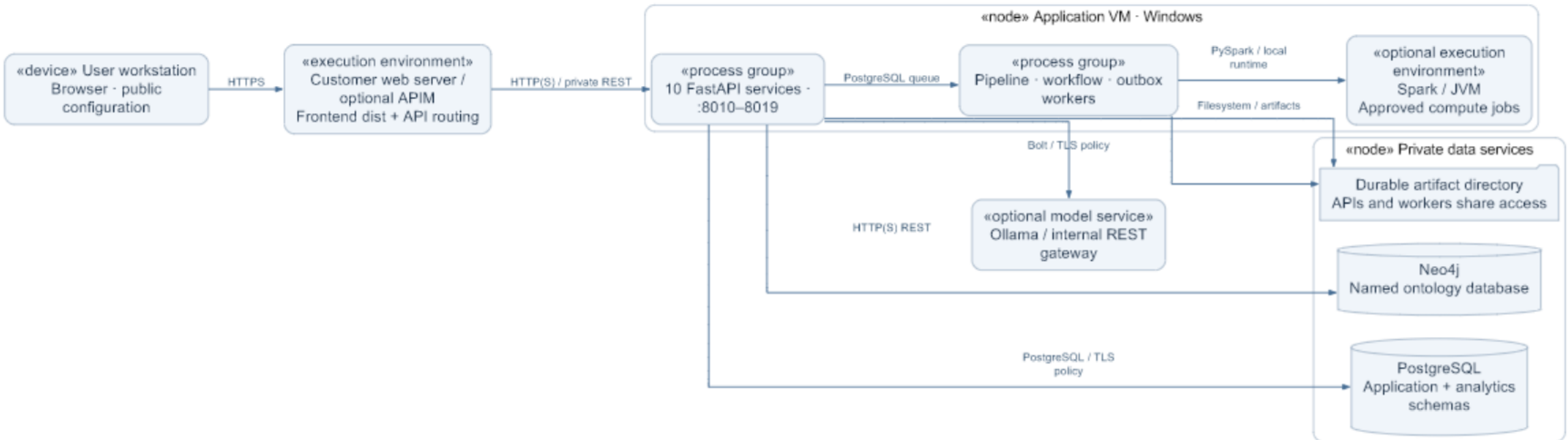
WHERE Private Graph API and named Neo4j database.

WHO Ontology, CEIM, Agentic and Graph services.

HOW Scoped credentials, batched writes and receipts.

UML deployment — customer runtime and network zones

The deployment can use direct service URLs or a configured gateway.



WHAT Browser, ingress, APIs, workers and data services.

WHY Expose the UI while containing privileged data access.

WHO Customer operations and service identities.

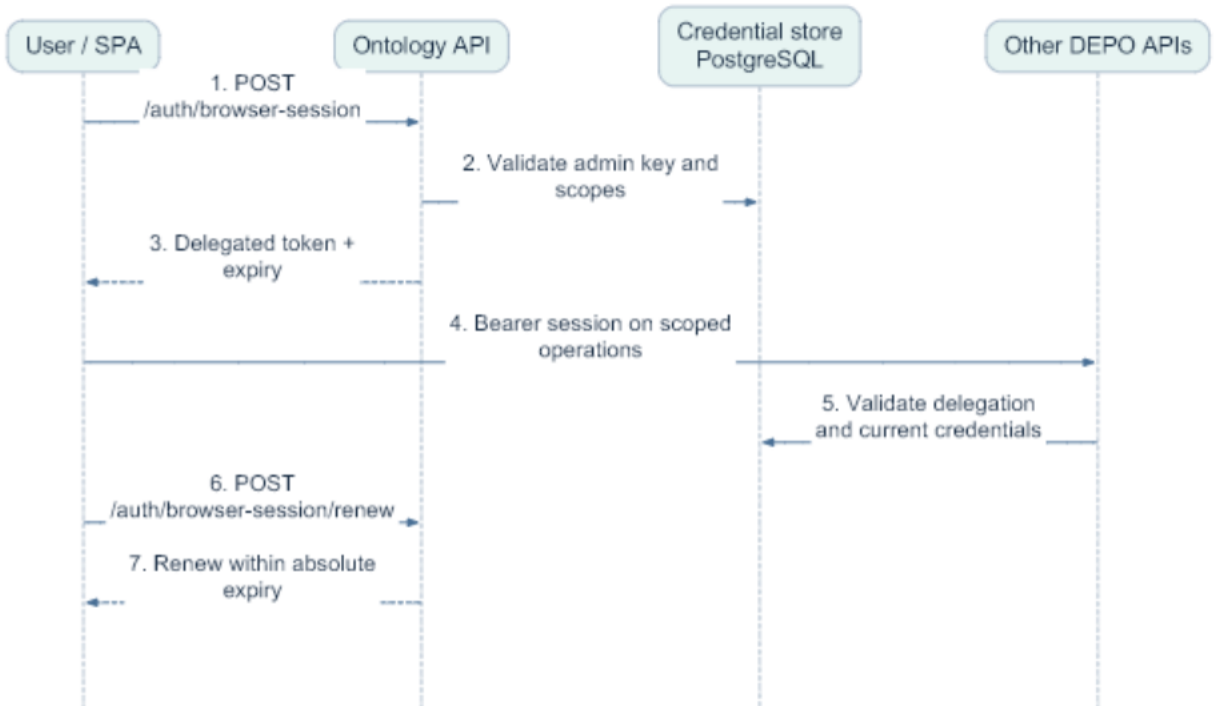
WHEN Installation, normal operation and recovery.

WHERE Application VM and private database network.

HOW Reviewed routing, TLS policy and durable storage.

UML sequence — delegated browser access

Reconnection restores access; expiry does not undo completed writes.



WHAT A bounded browser session with selected scopes.

WHY Avoid repeated raw-key entry during active use.

WHO User, ontology session API and protected APIs.

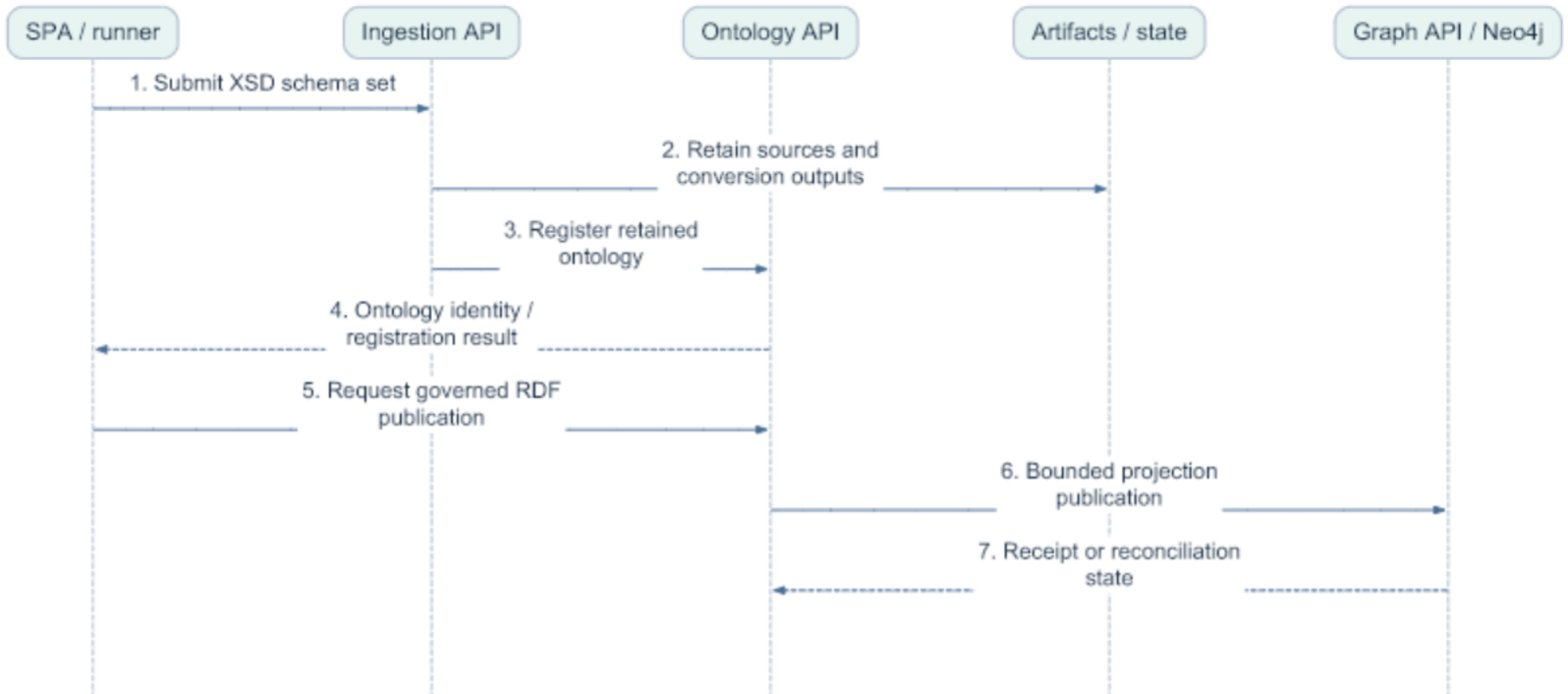
WHEN Connect, authorized requests and active renewal.

WHERE Browser token; PostgreSQL credential validation.

HOW Admin verification, delegation and absolute expiry.

UML sequence — source registration and RDF publication

A registered ontology and a published RDF projection are separate states.



WHAT Create an ontology from retained engineering inputs.

WHY Separate registration evidence from graph availability.

WHO User, ingestion, ontology and graph services.

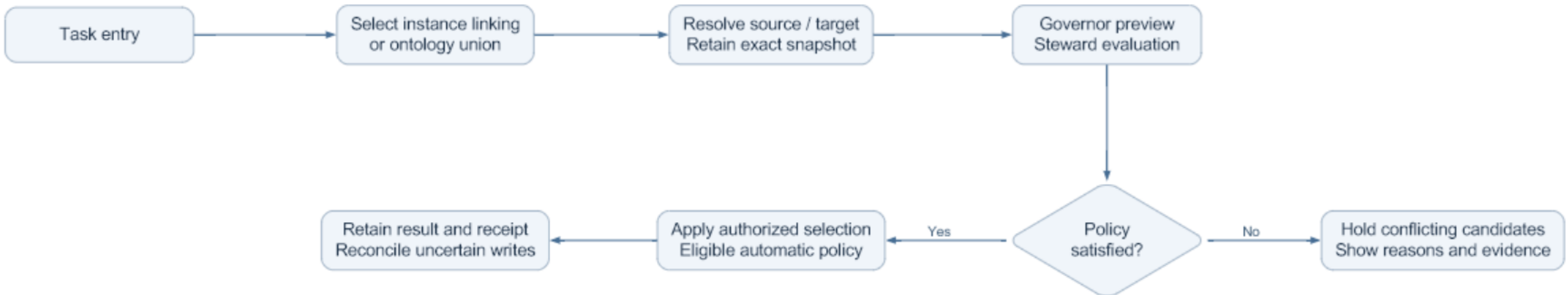
WHEN Schema conversion, then authorized publication.

WHERE Artifacts and PostgreSQL; RDF projection in Neo4j.

HOW Retained identity, publication job and receipt.

UML activity — Semantic Bridge decision flow

Automatic policy routes hold unsupported, stale or conflicting candidates.



WHAT Instance linking or ontology-to-ontology union.

WHY Keep task types and conflict decisions explicit.

WHO Governor, steward, authorized user and services.

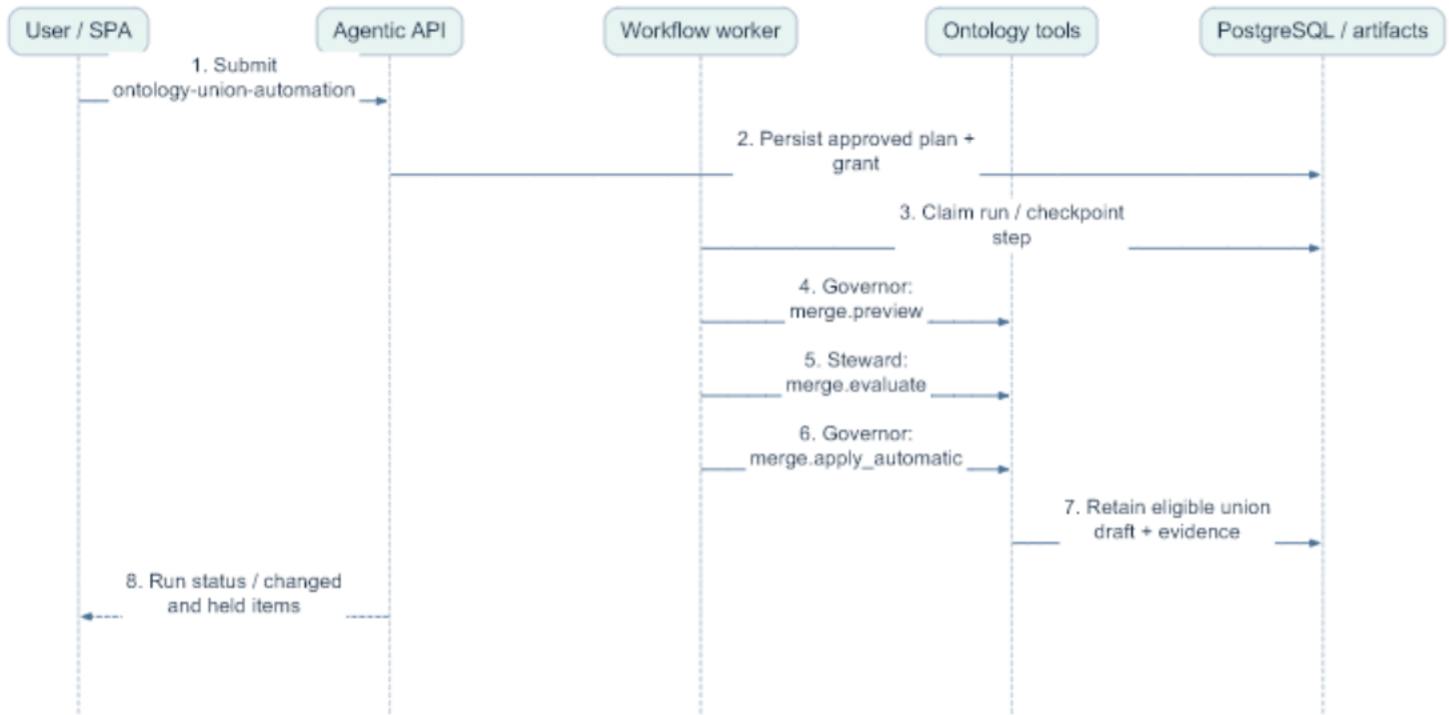
WHEN After exact source and target selection.

WHERE Ontology Junction / Semantic Bridge and workers.

HOW Preview, policy evaluation and retained outcomes.

UML sequence — automated ontology union draft

The workflow creates an eligible union draft; graph publication is a separate outcome.



WHAT A retained union draft with evaluation evidence.

WHEN Authorized ontology-union-automation run.

WHY Reduce repetitive review without bypassing policy.

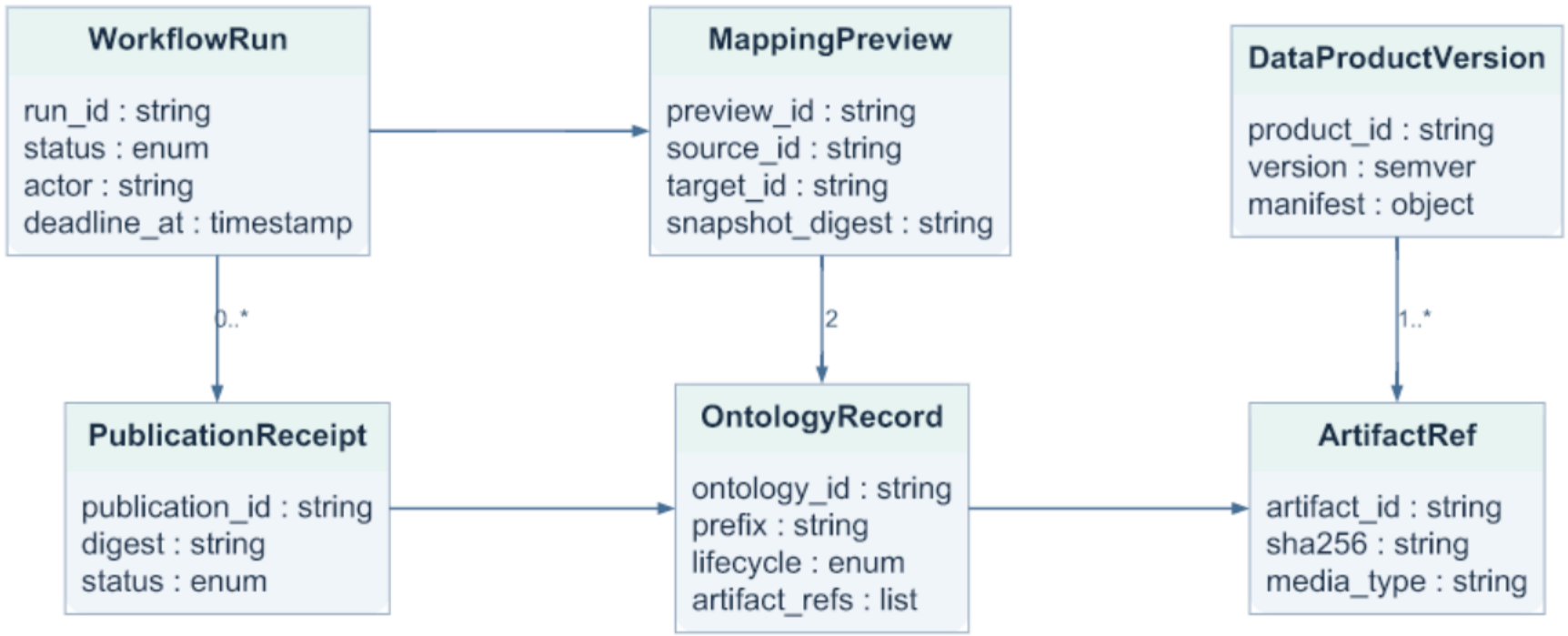
WHERE Agentic worker, ontology tools and retained stores.

WHO Governor previews; steward evaluates; governor applies.

HOW Checkpointed preview, evaluate and eligible apply.

UML class view — retained domain identities

Logical domain model · records may be JSON documents rather than ORM classes.



WHAT Logical records and their evidence relationships.

WHY Keep source, run, publication and product identities distinct.

WHO Domain services own records; clients use references.

WHEN Across review, execution and publication.

WHERE PostgreSQL records plus referenced artifacts.

HOW Stable keys, digests and explicit cardinalities.

Data ownership — application state, ODS, RDF and artifacts

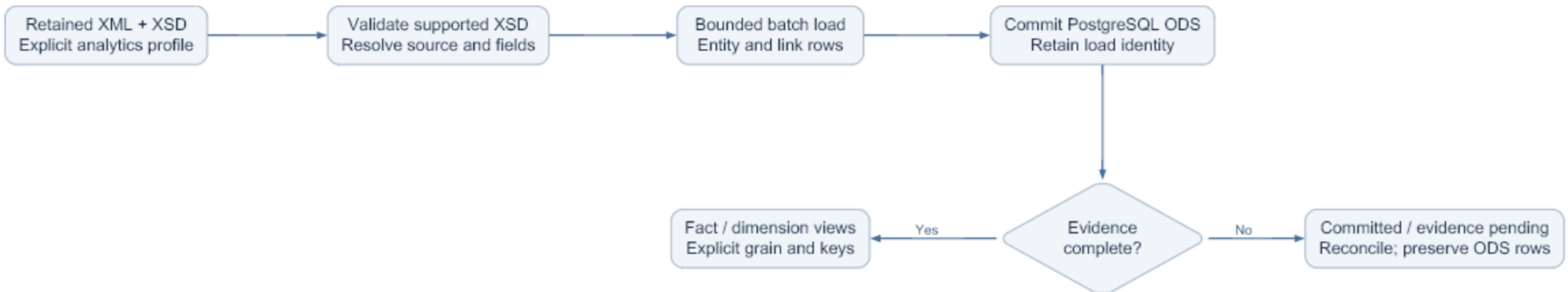
Storage responsibilities follow independent commit and publication boundaries.

Store	Owned data	Important boundary
PostgreSQL application schema	10 tables and 1 analytics view; registry namespaces, credentials and runs	Migrations 001–011; namespaces share relations
PostgreSQL analytics schemas	One versioned schema per retained XSD closure / prefix; XML entities and links	Fact / dimension views require explicit grain and keys
Neo4j	Published RDF resources, semantic relationships and publication evidence	Registration alone does not establish publication
Durable artifact storage	Original bytes, RDF, reports, load receipts and product packages	Retention and access policies apply to each artifact

A product package references selected evidence; it does not automatically export every ODS row or the entire graph.

UML activity — XML ODS and data-mart preparation

Business grain and history policy remain explicit; arbitrary files do not define a data mart.



WHAT Validated XML becomes structural PostgreSQL data.

WHEN Supported XSD projection and XML instance load.

WHY Keep a committed load recoverable after evidence failure.

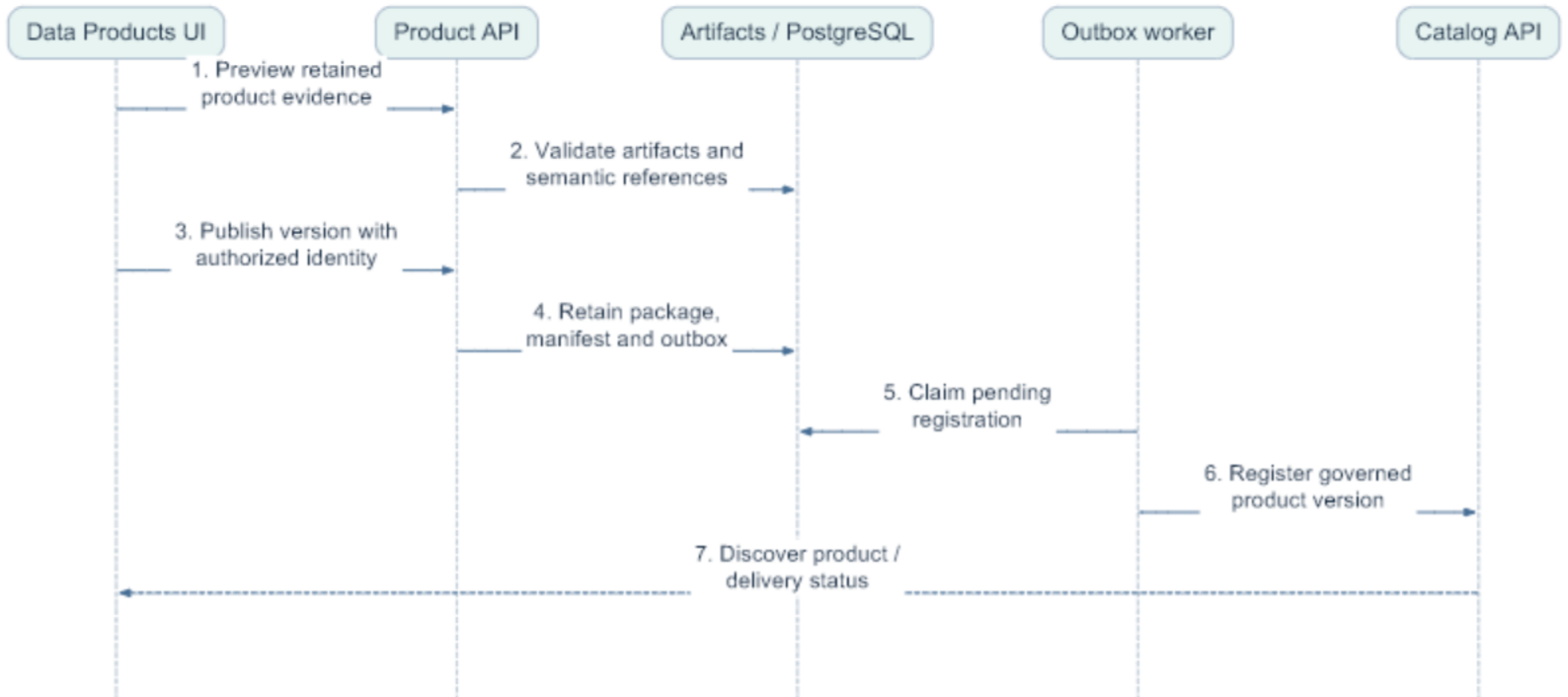
WHERE Versioned analytics schema in PostgreSQL.

WHO Pipeline worker and explicit analytics profile.

HOW Bounded batches, commit receipt and reconciliation.

Sequence — product packaging and catalog delivery

Catalog retries reconcile registration; they do not rerun ingestion.



WHAT A versioned package and discoverable catalog record.

WHEN After evidence and authorization validate.

WHY Separate durable packaging from delivery retries.

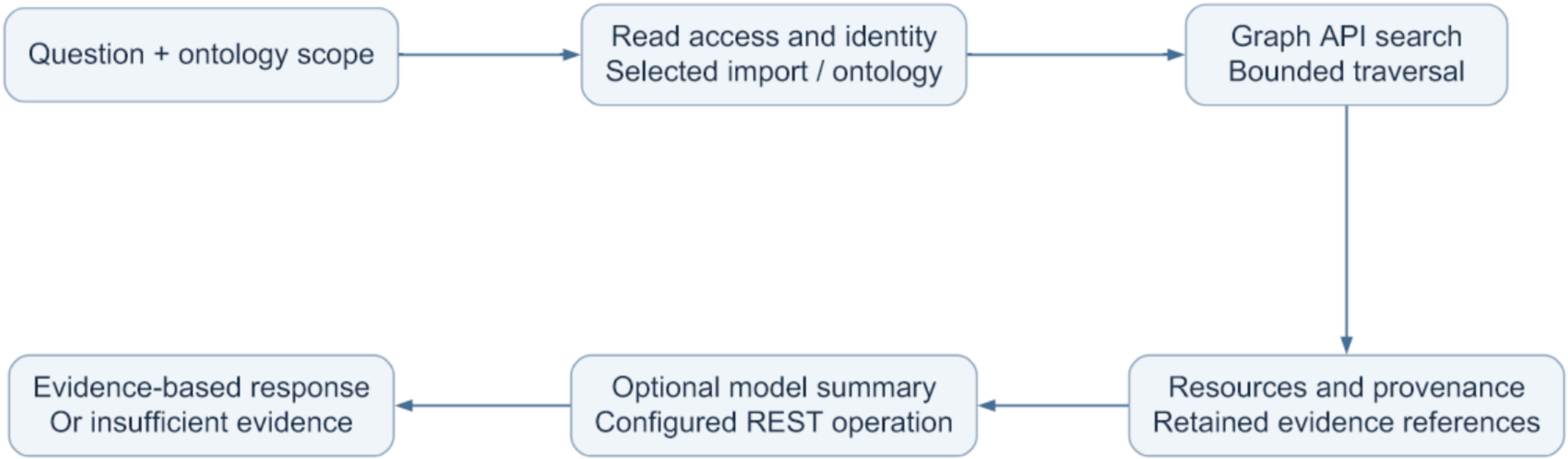
WHERE Artifact packages and PostgreSQL product state.

WHO Product API, outbox worker and catalog service.

HOW Preview, immutable package and outbox reconciliation.

Scoped graph retrieval and Knowledge Companion

Model availability and graph evidence availability are independent checks.



WHAT Bounded graph evidence and optional model summary.

WHEN A user submits an authorized scoped question.

WHY Explain answers using the selected ontology context.

WHERE Graph resources, provenance and response evidence.

WHO Knowledge Companion, Graph API and optional model.

HOW Search, bounded expansion and evidence references.

Ollama REST integration — verify capabilities separately

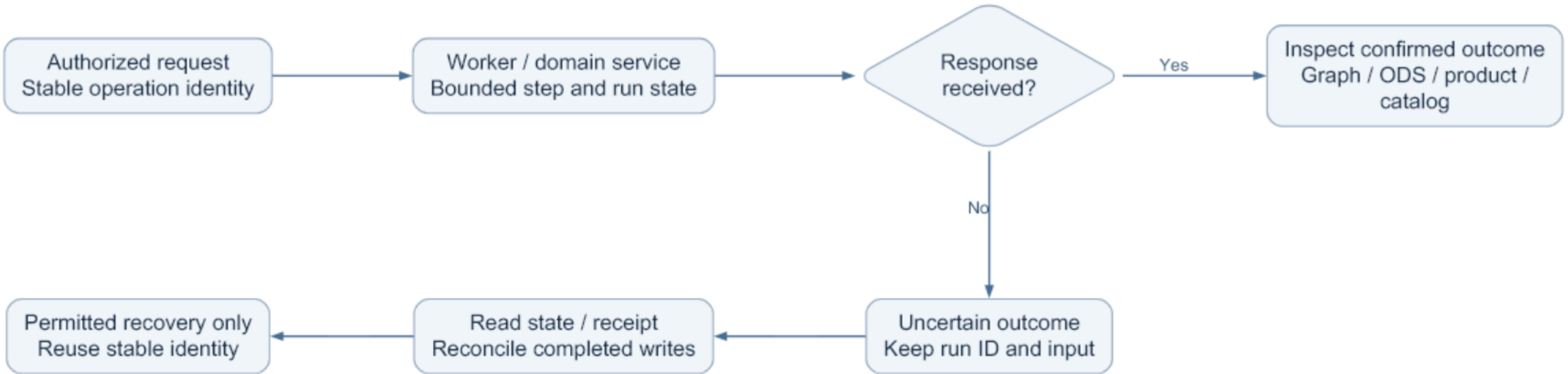
REST operation paths follow the deployed gateway contract.

Capability	Contract and control	Verification meaning
Generation / chat	Use the configured API root or explicit operation URL	A completed response verifies that operation only
Model discovery	GET /api/tags only when the gateway exposes it	A missing optional discovery route does not prove chat failure
Streaming	Validate native response frames and terminal completion	HTTP 200 alone does not establish a complete stream
Tool calling	Model / route capability check; tools retain their own authorization	Chat completion alone does not prove tool support
Credentials	Server-side API key header and explicit gateway scope	Never compile LLM or gateway keys into browser settings

Examples in configuration are guidance; test the actual customer route, model and response contract.

Failure lifecycle — uncertain response, retained outcome

Browser session expiry does not cancel an authorized background workflow.



WHAT Recovery across side effects and response loss.

WHY Prevent duplicate writes after a client timeout.

WHO UI, workers, domain APIs and operators.

WHEN Timeout, interruption or downstream failure.

WHERE Persisted run state and service-owned receipts.

HOW Inspect, reconcile and reuse stable identities.

Security and governance boundaries

Session lifecycle, operation authority and workflow execution are separate controls.

Boundary	Implemented control	Operating rule
Browser access	Delegated session, selected scopes, active renewal and expiry	Reconnect after full expiry / revocation; raw admin key stays out of builds
Protected service operations	Credential profiles and authenticated actor resolution	Model recommendations never grant approval authority
Durable workflow execution	Persisted grant, credential fingerprints, lease and checkpoints	No caller secrets in retained workflow inputs
Network and data services	Configured gateway roots, exact origins and database identities	Customer validates TLS, firewall access and least privilege
Operational maintenance	Separate admin authorization, scoped cleanup and audit evidence	Cleanup actions retain their own permissions and confirmations

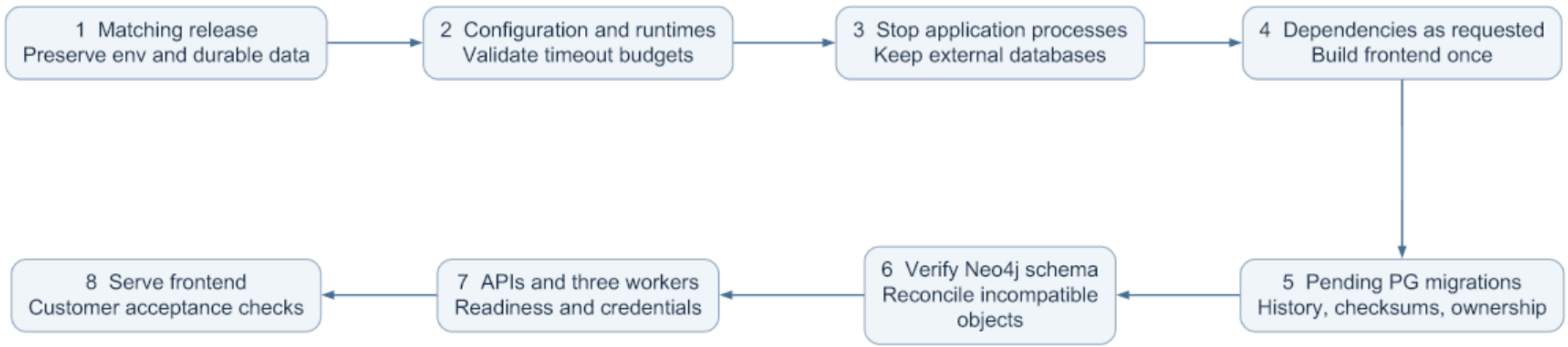
Installation entry points and database upgrade responsibilities

Python 3.12.x · Node.js 24+ · npm 10.2+ · Spark/JDK only for enabled compute paths.

Command / script	Owned responsibility	Existing customer installation
Root install-depo.ps1	Delegates to complete Windows installer	Use one entry point; do not run both installers separately
infra/windows/install-depo-windows.ps1	Dependencies, migrations, graph schema, startup and validation	Preserve environment files and durable data
infra/windows/install-depo.ps1	Runtime prerequisites and dependency installation helper	Complete installer calls this helper automatically
infra/postgres/ update-postgres-schema.ps1	Pending PostgreSQL migrations and schema verification	Schema-only operation; no application restart
infra/windows/update-depo-windows.ps1	Validated update sequence using the selected environment	Use -InstallDependencies when release locks change

UML activity — customer update and restart sequence

Service timeout limit: 300 seconds. Longer workflow and publication budgets have separate controls.



WHAT An in-place update using the selected environment.

WHEN After placing the complete matching release.

WHY Fail early and preserve existing data and credentials.

WHERE Customer application VM; existing databases.

WHO Deployment operator and service-owned scripts.

HOW Preflight, pending migrations, readiness and acceptance.

Service contracts and primary frontend consumers

OpenAPI 3.0.3 and service discovery describe operation ownership and authorization.

Service / internal port	API responsibility	Primary UI / caller
Schema sets :8010	Engineering schema inspection and conversion	Import / QIF
Ontology :8011	Registration, lifecycle, modeling and publication jobs	Ontology Junction / Admin
Agentic :8012	Agent catalog, tools, workflows and Companion	Semantic Bridge / Companion
Graph :8013	Publication receipts, scoped queries and traversal	Graph Explorer / domain APIs
Ingestion :8014	Sources, supported parsers and retained evidence	Import / engineering workflow
OSLC :8015 · CEIM :8018	Linked resources / canonical mapping and validation	Configured integrations / semantic jobs
Products :8017 · Catalog :8016	Immutable packages / governed product discovery	Data Products / Data Catalog
Pipeline :8019	Job definitions, worker runs and analytics recovery	Data Flow / XML analytics loader

Operational readiness and customer acceptance

Repository verification supports release preparation. Customer acceptance establishes deployment readiness.

Implemented controls

Durable run identities and receipts

Scoped access and request IDs

Bounded writes and recovery paths

Configuration and migration checks

Customer evidence

Database ownership and history

Gateway, TLS and browser access

Large-ontology load and recovery

Worker restart and backup restore

Explicit product choices

Fact grain and dimension policy

Named external source adapters

Mapping / retrieval evaluation

Model capability and deployment sizing

Architecture decisions and extension points

A framework replacement alone does not establish correctness, interoperability or customer readiness.

Stable service boundaries

Keep domain APIs as the source of operation contracts.

Expose governed tools through the existing catalog and MCP adapter.

Evidence before scale

Measure import, mapping and retrieval outcomes on agreed engineering data.

Preserve lineage and recovery identities when adding connectors.

Independent outcomes

Track ontology registration, RDF publication, ODS commit, product packaging and catalog delivery separately.

Use explicit business contracts for new analytics.